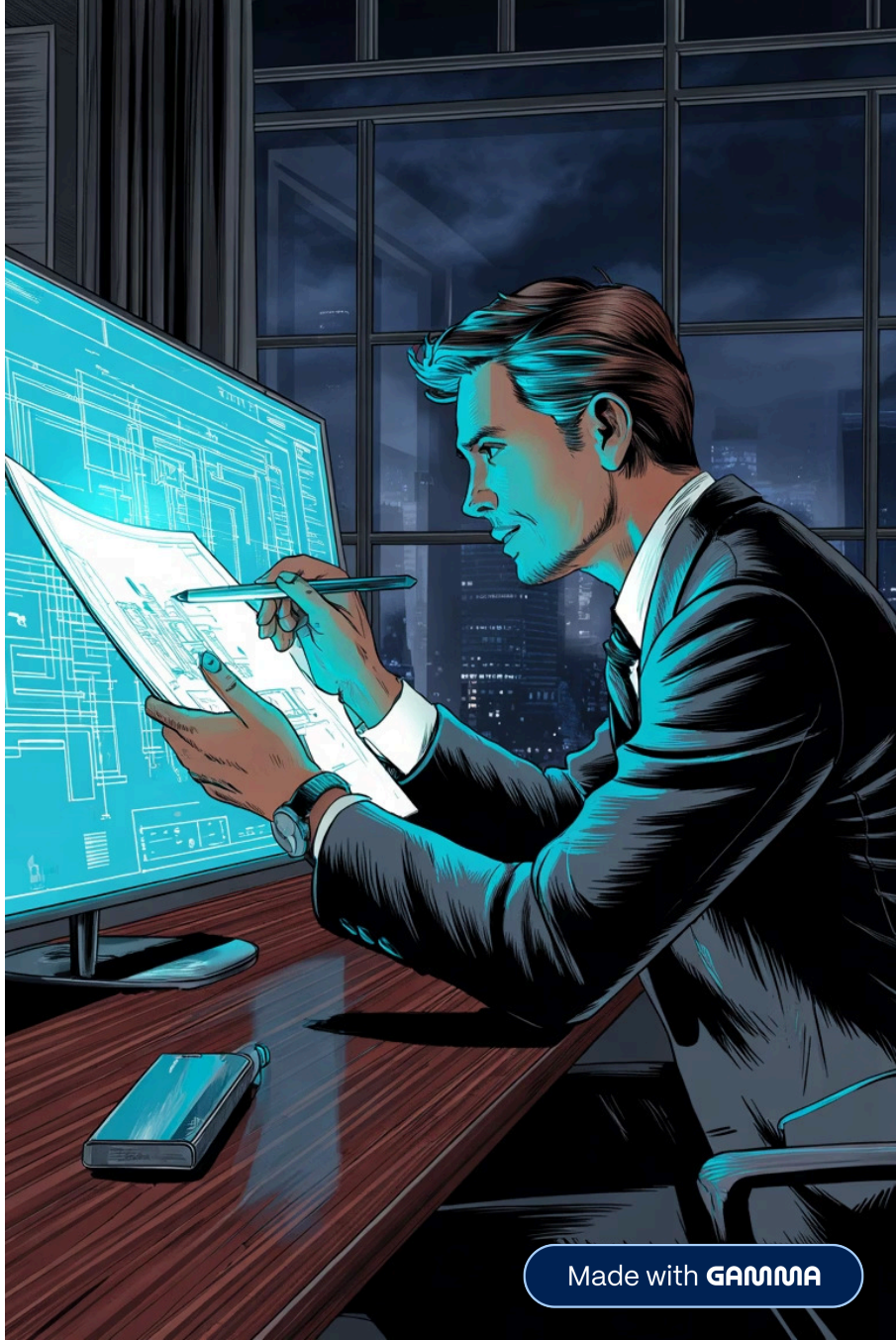
# System Architect: ScropIDS

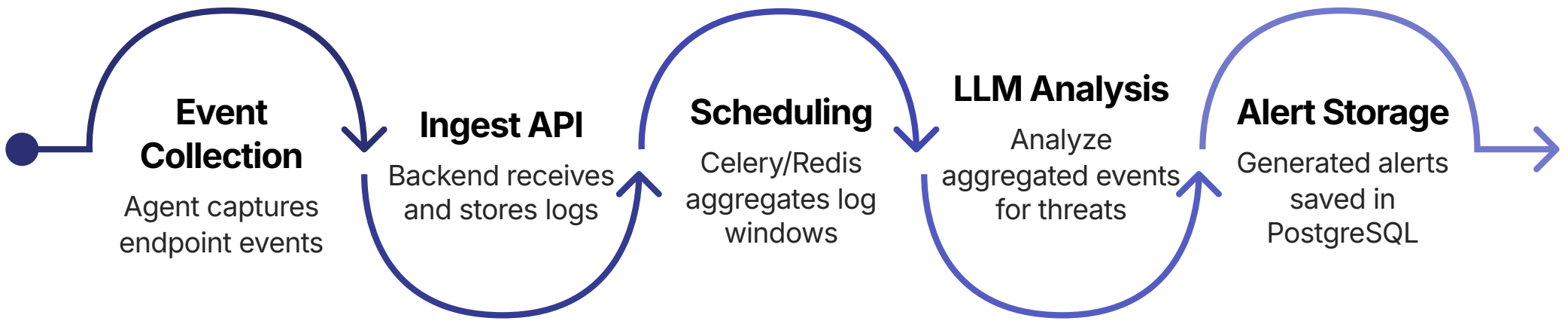
As the **System Architect** for ScropIDS, I was responsible for designing and unifying the entire technical structure of an AI-powered Intrusion Detection System. From endpoint agents to LLM-driven analysis and a React dashboard, I connected every layer — backend, frontend, scheduler, database, and AI workflow — into a single, coherent architecture. This presentation outlines the decisions, patterns, and principles that shaped the system.

AI-POWERED IDS | FULL-STACK ARCHITECTURE | SCROPIDS PLATFORM



# Architecture Planning & Data Flow

The core architectural challenge was defining a reliable, end-to-end data pipeline that could carry raw endpoint events all the way through to actionable security alerts. Every handoff between components was deliberately designed to be asynchronous, validated, and auditable.




This flow ensures that no component is tightly coupled to another. The scheduler acts as the orchestration layer, decoupling real-time ingestion from compute-intensive LLM analysis — keeping the system responsive under load.

# Backend & Data Design

The backend was architected around **seven core domain modules**, each with clearly scoped responsibilities. Multi-tenant organisation-based data separation was a foundational requirement — every query, every alert, and every agent registration is scoped to an organisation.

### Organisations

Tenant root entity. All data is isolated per organisation, enabling multi-client deployments from a single instance.

### Agents & Events

Endpoint agents register under an organisation. Raw events are timestamped, validated, and linked to their source agent.

### Aggregated Windows

The scheduler groups events into time-windowed batches — the unit of input for LLM threat analysis.

### Alerts & Config

Structured alert records with severity levels. LLM Provider Config and Scheduler Config allow per-org tuning without code changes.

# Security & Reliability Decisions

Architecture is only as strong as its security model. Every integration point in ScropIDS was designed with explicit threat assumptions — from agent communication to LLM response handling.

## Security Pillars

### → Token-Based Agent Auth

Each endpoint agent authenticates using a unique, rotatable token — preventing unauthorised log injection.

### → Organisation Isolation

Database-level tenancy ensures no cross-organisation data leakage, even under misconfigured queries.

### → Encrypted API Key Storage

LLM provider credentials are stored encrypted at rest, never exposed in logs or serialised responses.

## Reliability Patterns

### Strict JSON Validation

All LLM responses are parsed against a defined schema. Malformed or unexpected outputs are rejected and logged — preventing silent failures from propagating to alerts.

### Threshold-Based Alert Generation

Alerts are only created when LLM-assessed threat scores exceed configurable thresholds per organisation, reducing noise and false positives.

### Async Scheduling via Celery/Redis

Decoupled task execution ensures the backend API remains responsive even during heavy aggregation or LLM call cycles.

# Final Contribution

> "My role was to turn the project idea into a working technical blueprint."

As System Architect, my contribution was to ensure that every team member's work — from the agent developer to the frontend engineer — had a clear, stable foundation to build upon. I defined the interfaces, the data contracts, and the integration points so that components could be developed in parallel without conflict.

### Scalable by Design

Modular domain separation and async processing allow the system to scale horizontally across organisations and agent volumes.

### Team Integration

Clearly defined API contracts and module boundaries enabled parallel development — each contributor knew exactly where their work connected.

### Production-Oriented

Security, tenancy, and reliability patterns were built into the architecture from day one — not retrofitted — making ScropIDS ready for real-world deployment.